ReplayMark: Probe-and-Replay Model-Response Watermarking for Diffusion Language Models

Chia-Ming Lee[1] **Wen-Hsin Tsai**[2] **Tung Yuk Ying**[2] **Chih-Chung Hsu**[1]

[1]National Yang Ming Chiao Tung University [2]National Cheng Kung University

Abstract

Text watermarks usually hide a secret in token choices and later test those tokens without rerunning the generator. We ask whether a masked diffusion language model (DLM) can instead help verify its own output. Hiding selected parts of the context changes the probability of an emitted token in a reproducible, checkpoint-specific direction. ReplayMark uses that model response as the watermark carrier. The method first finds positions where plausible candidates exhibit both response directions. Only after a position is fixed does the key choose which direction to favour. The generator then resamples from its current distribution without changing the decoding schedule. To verify a finished document, ReplayMark reconstructs the same masked contexts and counts how often the responses agree with the key. Position selection cannot observe the requested direction, so text produced independently of the key gives an exact binomial count. At 1% FPR, ReplayMark reaches 0.80 TPR on LLaDA-8B-Instruct and 0.90 on Dream-7B-Instruct. The strongest token-local baseline reaches 0.93 on LLaDA but 0.57 or less on Dream when its LLaDA setting is transferred unchanged. At ReplayMark's main setting, the PPL ratio is $0.74\times$ on LLaDA and $1.09\times$ on Dream; across greedy and multinomial task runs, its largest absolute change from a paired control is 0.10. This checkpoint binding has a cost: verification needs the same model and 144 full-sequence calls per 512-token document, and dispersed edits reduce detection power. Code is available at `https://github.com/ming0531/ReplayMark`.

1 Introduction

When language-model text resembles human writing, its origin can be difficult to establish in settings such as disinformation analysis, phishing investigation, and academic-integrity review [25]. A post-hoc detector learns recurring output patterns; a watermark instead changes generation under a secret key and lets the key holder test a later document [18, 30]. We focus on watermarking because it can offer a stated finite-sample false-positive rate, rather than relying on a classifier whose behaviour may change with the decoder [9, 16, 20, 41].

Most language-model watermarks encode the secret in token identities by biasing a vocabulary subset or coupling randomness to each draw. Their detector needs only the text and key (Figure 1a) [18, 21]. This inexpensive design must coexist with the decoder. Masked DLMs resolve positions in parallel and order commits by confidence, so a forced prefix or altered order changes the system being watermarked [14].

DLM properties used. Re-masking, rather than the diffusion trajectory, provides:

- **Arbitrary-mask prediction:** one pass returns conditionals at every masked position.
- **Context response:** different hidden prefix subsets yield checkpoint-specific probability shifts.
- **Replayability:** the same checkpoint reproduces those shifts from re-masked finished text.

Preprint.

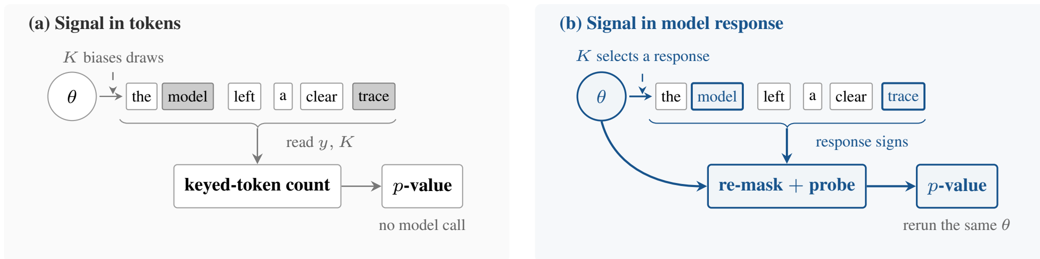

Figure 1: **Two places to encode a watermark.** Token-local methods alter sampled identities and verify from y, K alone. ReplayMark instead encodes which context response the checkpoint favours, so verification reruns the same θ.

Two obstacles separate this observation from a valid detector. First, verification must recreate the generation-time state even though the full continuation is now visible. Second, the method may seek positions where both response directions are plausible, but it must finish that search before the key reveals which direction counts as a match. Reversing this order would bias the selected positions toward the requested answer.

ReplayMark addresses both obstacles. For each active block, it re-masks that block and its suffix, compares pairs of hidden-prefix probes, and selects positions whose smaller directional probability total exceeds a fixed cutoff. The key then supplies the requested direction. At a selected position, ReplayMark draws up to R candidates from the decoder's current distribution and commits the first match; all other positions follow the reference decoder. Verification reconstructs the probes, repeats the selection rule, and evaluates an exact binomial tail. The block partition, confidence order, and checkpoint probabilities remain those of the reference decoder.

Our contributions are:

- We introduce model-response watermarking: the signal is a checkpoint's reproducible reaction to hidden context rather than a keyed pattern in token identities.
- We give a replayable construction whose carrier positions are selected before the keyed orientation is consulted, yielding finite-sample false-positive control while preserving the reference decoding schedule (Sections 3.2–3.5).
- We evaluate two masked DLMs. At 1% FPR, ReplayMark reaches 0.80 TPR on LLaDA and 0.90 on Dream. Verification requires 144 full-sequence model calls, and detection power decreases under dispersed edits (Section 4).

2 Related Work

Masked diffusion language models. Masked DLMs generate text by repeatedly predicting unresolved positions and committing a subset of them. This extends discrete diffusion to text [3, 13, 26, 29, 34, 35] and now supports instruction-following models including LLaDA and Dream [22, 28, 39, 42]. Which positions commit, and in what order, affects both quality and speed [2, 6, 17, 23, 36, 37]. We therefore treat the deployed schedule as a constraint: ReplayMark observes the model's masked-token predictions but does not choose a new commit order.

Watermarks encoded in tokens. Autoregressive watermarks usually bias a keyed vocabulary subset [18] or couple each draw to keyed randomness [1, 11, 21]. Their detectors are cheap because a key–token calculation needs no model call, and later work characterises when those detectors are reliable or undetectable [9, 19, 41]. DLM methods adapt this idea by removing prefix dependence, averaging a red–green bias over unresolved context, sampling with keyed randomness, or encoding information in the denoising order [4, 8, 12, 14, 31, 38]. Our closest baseline, dgMARK, changes which mask resolves next and later reads that order from the text. ReplayMark instead keeps the order fixed and asks the checkpoint to reproduce its response.

Detectors that use a model. Model access can support richer tests than a key–token hash. Likelihood-ratio detectors compare watermarked and base conditionals [15]; access to the model can improve a Neyman–Pearson score over a key-only Gumbel statistic [10]; GaussMark verifies a keyed weight perturbation with a forward–backward pass [7]; and model-theft watermarks query a suspect system [40]. Outside watermarking, DetectGPT scores text using log probabilities from the model of interest and perturbed versions of the passage; replacing the source model with a surrogate weakens detection [27]. These model-assisted detectors are most natural when a provider or central verifier can afford model access. ReplayMark shares that operational setting but adds a keyed generation channel and a document-level test. It leaves the checkpoint unchanged and keys the contexts presented to it, so one deployed model can serve many keys.

Finite-sample detection and editing. Watermark tests increasingly replace asymptotic z-scores with exact tails, permutation tests, or corrected maxima over windows [10, 19, 21]. After editing, a statistic that concentrates on surviving regions can outperform a document-wide sum [24]. ReplayMark uses an exact binomial tail for the unedited detector and compares document-wide, block-local, and windowed aggregation under editing in Section 4.2.

3 ReplayMark: Watermarking by Model Response

3.1 Masked Decoding

A masked DLM predicts unresolved positions in parallel rather than exposing a unique "next token." Let x be the prompt, $y = (y_1, \dots, y_N)$ the continuation, and I the revealed positions. At each $i \notin I$, one full-sequence pass returns $p_\theta(y_i \mid y_I, x)$; we count that pass as one full-sequence model call.

We study LLaDA's blockwise schedule [2, 28]: blocks finish left to right, while confidence determines the within-block commits. We write B_b for the active block and F_b for its completed prefix. ReplayMark preserves both ordering rules and changes only the auxiliary mask patterns used to query checkpoint probabilities.

3.2 Problem Setup

We frame the task as key-specific provenance: given a completed prompt–continuation sequence, decide whether it was produced through the keyed procedure, rather than merely whether it resembles output from the model. The false-positive level is fixed before inspecting the text. The watermark may change a sampled token, but not the model's decoder or schedule.

What the two parties retain. The embedder has the model, prompt, secret key K, and target bits. The verifier later receives the completed sequence, same checkpoint, K, and public parameters, but no trace, logits, seed, or rejected samples. It must therefore recreate each query from the submitted text rather than consult a stored sampling record. A nonce gives each document a fresh derived key, $K_d = \mathrm{HMAC}(K, \mathrm{nonce}_d)$ [5]; separate hash domains cover probes, carriers, and response directions.

Guarantee and scope. For text chosen without knowledge of K_d (including human text and generations from another sampler or an independent document key), a detector run at level α fires with probability at most α; no source distribution is needed. The probability is over the secret keyed response directions. Because carrier selection does not consult them, the exact test remains valid with a data-dependent carrier count. This finite-sample statement controls false attribution, not robustness. It does not promise detection after editing, key disclosure, adaptive detector queries [16], or replay with a numerically different checkpoint.

Why write during generation? Editing only after completion offers too little freedom at most positions. Across 36 post-hoc substitution settings, the median selected position has only one candidate within a 3-nat likelihood budget; the best setting below a $1.35\times$ PPL budget reaches 0.10 TPR at 1% FPR. ReplayMark therefore writes while the model's live distribution is still available.

The design follows three requirements:

1. **Replayability.** Reconstruct the generation-time masked context from finished text.
2. **Blind selection.** Fix carriers before the key reveals the matching response direction.
3. **Schedule preservation.** Keep the reference block and within-block commit order.

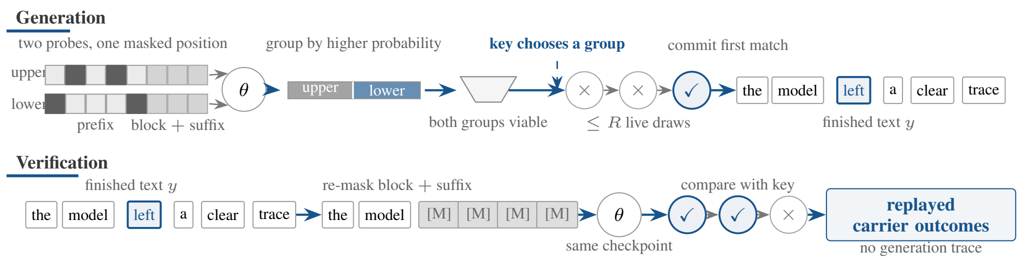

Figure 2: **Probe pair and replay.** Two hidden-prefix probes split candidate tokens by which probe assigns more probability. Both groups must be viable before the key chooses one; verification recreates the same probes from the finished text.

End-to-end procedure. These requirements lead to five operational steps:

1. **Initialise.** Register the checkpoint and schedule; derive the document key; fix L, R, κ, and α.
2. **Probe.** Use the DLM's arbitrary-mask conditionals to compare keyed prefix subsets.
3. **Write.** Select the carrier before its keyed direction; commit the first plausible match within R draws, otherwise take a fresh draw.
4. **Replay.** From the finished text, key, and same checkpoint, reconstruct contexts and carriers.
5. **Decide.** Count matches and detect when the exact binomial p-value is at most α.

Figure 2 illustrates Steps 2–4; the following subsections define the three rules.

3.3 Probing the Model and Selecting Carriers

Steps 2–3 split candidates by which masked-prefix probe assigns more probability. A position is usable only when both groups have sufficient live probability; the key then chooses which group the emitted token should favour.

Formally, ReplayMark constructs L pseudorandom probe patterns $D_{b,\ell} \subset F_b$, where each pattern specifies prefix positions to hide. It also masks the current block B_b and every later position. Let $\lambda_{i,\ell}(v)$ denote the resulting log-probability of vocabulary item v at position i under pattern ℓ. The block and suffix are hidden because neither was available when the probe was created. The verifier can therefore reconstruct the same state from the completed sequence instead of seeing later tokens that would alter the probabilities. Probing a block costs $L+1$ full-sequence model calls: L masked-prefix probes and one evaluation with the full prefix visible.

For a pair (ℓ, ℓ'), define the response $g_i^{\ell,\ell'}(z) = \lambda_{i,\ell'}(z) - \lambda_{i,\ell}(z)$. Let p_i^{base} be the block-masked distribution with no hidden prefix. The aggregate probability assigned to either response direction, and a symmetric score for the pair, are

$$q_{i,\pm}^{\ell,\ell'} = \sum_z p_i^{\text{base}}(z)\mathbf{1}[\pm g_i^{\ell,\ell'}(z) > 0], \qquad S_i^{\ell,\ell'} = 2\min(q_{i,+}^{\ell,\ell'}, q_{i,-}^{\ell,\ell'}). \tag{1}$$

The score is twice the smaller total. It approaches one when the live distribution is divided evenly between the two response groups and approaches zero when one group has little probability. At each position, ReplayMark keeps the pair with the largest S_i and selects the position when $S_i > s_{\min}$, subject to an optional per-block cap. Choosing the best pair raises the measured mean score from 0.28 for a fixed near/far pair to 0.45. Figure 3 shows where this criterion retains or skips positions in one saved LLaDA output.

Crucially, swapping ℓ and ℓ' leaves S_i unchanged while reversing every response sign. Carrier selection therefore cannot favour the direction later requested by the key.

After the carrier is fixed, the key assigns one group to the target bit. Formally, a separate hash domain supplies $\epsilon_i \in \{-1, +1\}$, while $w_i \in \{-1, +1\}$ represents the provenance or payload bit. Together they decide which response direction counts as a match:

$$m_i(z) = \mathbf{1}[w_i \epsilon_i g_i(z) > 0]. \tag{2}$$

If $g_i(z) = 0$, a second domain-separated bit assigns the outcome. Such a tie cannot be steered, but it remains an independent fair outcome for detection.

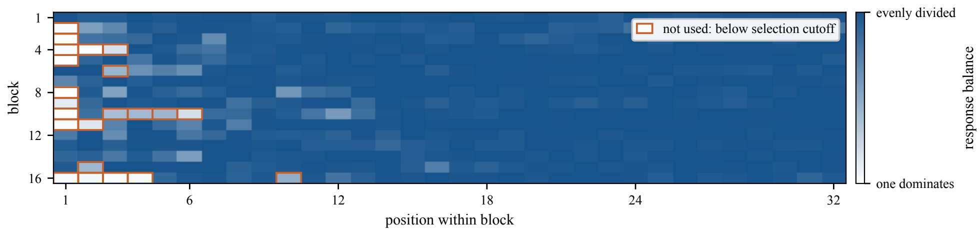

Figure 3: **Carrier availability in one generated document.** Each cell is one token position. Darker blue indicates greater balance between the response groups in Figure 2; orange outlines mark positions below the cutoff, where no bit is written.

3.4 Embedding Without Changing the Schedule

The reference decoder still chooses the next position and supplies its live distribution p_i. Unselected positions are sampled once as usual. At a selected carrier, ReplayMark accepts a candidate only if it has the requested response and is not too far below the most likely token:

$$A_i = \left\{ z : w_i \epsilon_i g_i(z) > 0 \text{ and } p_i(z) \geq \kappa \max_v p_i(v) \right\}. \tag{3}$$

It draws from the same p_i up to R times and commits the first member of A_i. If every proposal misses, it takes one fresh unchecked draw. All retries reuse p_i, so they add no model call and do not alter the commit order.

Let $a_i = p_i(A_i)$ be the probability of satisfying both conditions, and let $q_i = \sum_z p_i(z) m_i(z)$ be the chance that an unchecked draw scores as a match. The committed token comes from $p_i(\cdot \mid A_i)$ with probability $1 - (1 - a_i)^R$ and from p_i otherwise. Its match probability is therefore

$$\Pr[m_i = 1] = 1 - (1 - a_i)^R (1 - q_i). \tag{4}$$

With no plausibility floor and no ties, $a_i = q_i$, giving $1 - (1 - q_i)^{R+1}$. Equation (4) separates the two controls: R determines how many chances the sampler has to find an acceptable token, while κ limits how unlikely that token may be relative to the live argmax. Larger R can strengthen detection but changes the sampling distribution; κ does not constrain the unchecked fallback or guarantee semantic quality, which we measure separately.

3.5 Watermark Detection

The verifier re-masks each completed block and suffix, reconstructs the probe pairs and carriers, and evaluates $m_i(y_i)$. It needs no rejected samples or generation trace. For provenance detection, the selected carriers form a pool P with $w_i = +1$; a payload implementation would keep a separate pool for each message bit. The document statistic and its one-sided p-value are

$$T = \sum_{i \in P} m_i, \qquad p_{\text{det}} = \Pr\left[\text{Binomial}(|P|, \tfrac{1}{2}) \geq T\right]. \tag{5}$$

Figure 4 follows a five-carrier example from replayed outcomes to the exact upper-tail probability.

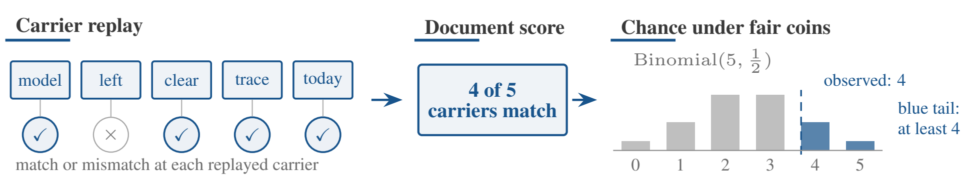

Figure 4: **Document-level detection.** The verifier replays each carrier and records a match. Here four of five match; the blue bars give the exact chance of observing at least four matches under fair coins.

Proposition 1 (Exact false-positive control)**.** Assume the hash domains behave as independent random functions. For any text written without the document key, condition on the selected carriers, probe pairs, responses, and target bits. The orientation bits remain independent fair coins. Hence the m_i are independent Bernoulli($\frac{1}{2}$) variables, $T \sim \text{Binomial}(|P|, \frac{1}{2})$, and thresholding Eq. (5) at α gives false-positive rate at most α for every document length and carrier count.

Method	Setting	PPL ratio ↓	TPR at FPR ↑		Full-sequence model calls ↓	
			1%	0.1%	Generation	Verification
LLaDA-8B-Instruct						
KGW [18]	$\delta = 1$	1.22×	0.90	0.83	512	**0**
dgMARK [14]	$k = 1$	1.45×	0.83	0.80	512	**0**
dgMARK [14]	+3-beam	1.21×	**0.93**	**0.93**	∼2048	**0**
ReplayMark	$R = 1$	0.97×	0.43	0.20	656	144
ReplayMark	$R = 8, \kappa = 0.1$	0.87×	0.70	0.60	656	144
ReplayMark	$R = 16, \kappa = 0.05$	**0.74**×	0.80	0.60	656	144
Dream-7B-Instruct						
KGW [18]	$\delta = 1$	1.84×	0.57	0.37	512	**0**
dgMARK [14]	$k = 1$	1.75×	0.37	0.27	512	**0**
dgMARK [14]	+3-beam	**0.71**×	0.30	0.30	∼2048	**0**
ReplayMark	$R = 1$	1.34×	0.40	0.27	656	144
ReplayMark	$R = 8, \kappa = 0.1$	0.79×	0.70	0.67	656	144
ReplayMark	$R = 16, \kappa = 0.05$	1.09×	**0.90**	**0.90**	656	144

Table 1: **Detectability and cost at 512 tokens.** PPL ratio is measured against each method's own unwatermarked run and should be interpreted within that method. ReplayMark uses $n = 30$ documents ($n = 20$ for $R = 16$); Dream PPL uses 8–13 successfully scored document pairs per setting. Settings chosen on LLaDA are transferred to Dream without retuning. One model call is one forward pass over the current full sequence; these counts measure algorithmic work, not runtime. δ is KGW's logit bias and k is dgMARK's candidate count.

Why adaptive carrier selection is allowed. The text and model may determine the number of carriers, the probe pair, and the response size. What matters is that each choice is fixed before the independent orientation bit is used. Flipping ϵ_i then flips the match whenever $g_i(y_i) \neq 0$; the tie rule supplies a fresh fair bit otherwise. The detector therefore needs neither an asymptotic approximation nor a calibration corpus. The guarantee still assumes text produced independently of the key and domain separation among the key's several roles.

4 Experiments

The evaluation asks four questions: detection strength, transfer across checkpoints, computational and quality cost, and behaviour under editing.

Models and data. We evaluate LLaDA-8B-Instruct [28] and Dream-7B-Instruct [39] in FP16 on one TITAN RTX. Detectability uses C4 `realnewslike` prompts [33], truncated to 300 characters, and 512-token continuations divided into 32-token blocks. We correct Dream's one-position logit offset in its wrapper, then use the same position convention for generation and replay.

Decoders and settings. ReplayMark and dgMARK use each model's confidence schedule with multinomial sampling at temperature 0.8; dgMARK additionally uses top-$k = 3$. KGW requires strictly left-to-right decoding. ReplayMark uses $L = 8$ probe patterns, threshold $s_{\min} = 0.5$, retry budget R, and optional plausibility floor κ. We evaluate provenance ($w_i = +1$), not payload recovery. Here one model call means one forward pass over the current full sequence. With 16 blocks and $L + 1 = 9$ probe evaluations, verification replays $16 \times 9 = 144$ calls; generation uses 512 ordinary decoding calls plus the same 144 probe calls, for 656 in total.

Metrics. Detection is TPR at the analytic FPR thresholds in Eq. (5). Perplexity is scored by GPT-2-large [32] as $\exp(\mathrm{mean}(\mathrm{NLL}_{\mathrm{wm}} - \mathrm{NLL}_{\mathrm{unwm}}))$. A PPL ratio therefore compares a method with its own unwatermarked decoder; it is not a semantic-quality score and is only indicative across different decoders. The baselines are KGW [18] and dgMARK [14].

4.1 Main Results and Analysis

Table 1 shows the tradeoff: ReplayMark transfers strongly to Dream but replay is costly, while two cheaper baselines detect LLaDA output more often.

Model	Method	Greedy decoding			Multinomial decoding		
		MMLU Acc ↑	GSM8K Acc ↑	HumanEval Pass@1 ↑	MMLU Acc ↑	GSM8K Acc ↑	HumanEval Pass@1 ↑
LLaDA-8B-Instruct	Non-watermarked	0.605	0.740	0.409	0.595	0.640	0.335
	KGW [18]	0.615	0.680	0.384	0.535	0.740	0.293
	dgMARK [14]	0.595	0.660	0.396	0.530	0.660	0.341
	dgMARK +3-beam	0.595	0.620	0.329	0.530	0.660	0.280
	ReplayMark	0.590	0.680	0.335	0.575	0.700	0.384
Dream-7B-Instruct	Non-watermarked	0.740	0.840	0.549	0.695	0.660	0.427
	KGW	0.680	0.760	0.524	0.625	0.700	0.421
	dgMARK	0.740	0.720	0.530	0.570	0.560	0.421
	dgMARK +3-beam	0.745	0.580	0.470	0.645	0.560	0.329
	ReplayMark	0.745	0.700	0.470	0.710	0.680	0.439

Table 2: **Task accuracy under greedy and multinomial decoding.** The Non-watermarked row provides a compact reference for each decoding half. Because implementations differ, each watermarked score should be interpreted against its paired run using the same decoder and harness, rather than by directly subtracting the displayed row. ReplayMark uses $R = 16, \kappa = 0.05$; its greedy reference commits use argmax while carrier proposals remain stochastic. MMLU, GSM8K, and HumanEval use 200, 50, and 164 problems, respectively.

Retries trade sampling distortion for detection. On LLaDA, $R = 1$ reaches 0.43 TPR at 1% FPR. On 30 held-out prompts, $R = 2, 4, 8$ increase TPR to $0.73, 0.83, 0.87$, consistent with Eq. (4). Retries add no model evaluation, but without a floor the PPL ratio rises from $1.14\times$ at $R = 2$ to $1.61\times$ at $R = 8$. The floor removes low-probability guided draws: $(R, \kappa) = (8, 0.1)$ gives 0.70 TPR at $0.87\times$ PPL, and $(16, 0.05)$ gives 0.80 at $0.74\times$. Ratios below one mean only that GPT-2-large prefers the retained high-probability tokens.

The transferred setting behaves differently across checkpoints. We carry every LLaDA setting to Dream without retuning. KGW reaches 0.57 TPR at 1% FPR and produces 84% duplicate bigrams under forced left-to-right decoding. Direct and three-beam dgMARK reach 0.37 and 0.30 TPR. ReplayMark at $(16, 0.05)$ reaches 0.90 at both reported FPR levels. Its PPL ratio is $0.79\times$ at $(8, 0.1)$ and $1.09\times$ at $(16, 0.05)$, based on 8–13 scored pairs per setting. Thus detectability transfers in this experiment, whereas the setting with the lowest LLaDA PPL ratio does not retain that ratio on Dream.

Task accuracy exposes decoder-specific costs. Table 2 reports both decoding regimes. Under greedy decoding, MMLU changes are small. On GSM8K, ReplayMark matches its paired LLaDA run (0.680 versus 0.680) and changes from 0.800 to 0.700 on Dream; with 50 problems, the Dream difference is not statistically distinguishable. The $+3$-beam variant changes from 0.840 to 0.580. On HumanEval, ReplayMark drops by 0.061 and 0.079, comparable in scale to $+3$-beam. Under multinomial decoding, ReplayMark changes by at most 0.06, whereas dgMARK variants incur larger Dream drops: 0.17 on MMLU, 0.28 on GSM8K, and up to 0.238 on HumanEval. On 256-token LLaDA GSM8K answers, ReplayMark and direct dgMARK reach 0.12 TPR at 1% FPR, three-beam dgMARK 0.44, and KGW zero; 512-token rates therefore do not transfer to short answers.

Checkpoint binding costs model calls. On LLaDA, ReplayMark's $0.80/0.60$ TPR at $1\%/0.1\%$ FPR trails three-beam dgMARK's $0.93/0.93$ and KGW's $0.90/0.83$, while requiring 144 verification calls instead of zero. KGW's left-to-right decoder also repeats heavily: 37% of unwatermarked documents cross the threshold before bigram deduplication, and two of thirty cross 0.1% afterward. With 20–30 documents, TPR differences near 0.1 remain imprecise. Wall-clock, verification takes $26.8\,\mathrm{s}$ per document on one TITAN RTX (KGW $0.17\,\mathrm{s}$, dgMARK under a millisecond); the cost is a dial rather than a fixed price, since $L = 4$ probe patterns cut the replay to 80 calls at 0.70 TPR at 1% FPR against 0.77 at $L = 8$, while $L = 2$ saves only 32 further calls and falls to 0.57 (Appendix Table 4).

4.2 Calibration and Robustness

Observed false-positive rates are below nominal levels. Across 20 keys and 120 unwatermarked LLaDA generations per key, mean FPR is $0.0121/0.0037/0.0000$ at nominal $0.10/0.05/0.01$; the largest per-key rates are $0.0583/0.0250/0.0000$. Every Dream cohort has zero false positives at 1%

(Appendix Table 3). The rule is calibrated on human text as well: over 1800 C4 documents (three keys), empirical FPR is $0.043/0.0072/0.0011$ at nominal $0.05/0.01/0.001$.

Dispersed edits break prefix synchronisation. At $R = 16, \kappa = 0.05$, random 5% and 10% edits leave 0.88 and 0.50 TPR, while a contiguous 10% or 30% span leaves 0.88 because earlier carriers survive. Under random 10% editing, pooled counting reaches 0.50, versus 0.38 for blockwise Bonferroni and 0.44 for 128-token windowing (Appendix Table 5). Pooled counting holds 0.81 after half the document is re-denoised; at 75%, it ties Bonferroni at 0.50. The same attack barely moves token-local detectors (KGW 0.90 to 0.85, dgMARK 0.85 to 0.80 at 1%, n=20), isolating prefix-synchronised replay as the weakness. Full-document Qwen2.5-7B paraphrase removes detection (0.00 from 0.88), the endpoint of the same desynchronisation.

5 Discussion

Operational fit.

- **Suitable:** disputes about a registered checkpoint, where a provider or auditor retains the model and the decision justifies replay.
- **Poor fit:** browser-scale classification, unknown models, short fragments, or heavily edited text; token-local detection is better suited to routine cases.

Who runs replay?

- **Provider service:** retain the registered checkpoint and return a signed replay report.
- **Independent auditor:** run the same procedure with escrowed weights.

Both avoid model deployment on the client. Batching helps, but document-specific contexts prevent a fixed lookup; surrogate checkpoints or cached responses require separate analysis.

Conditions for extension. Beyond the evaluated checkpoints, the method requires reproducible arbitrary-mask conditionals, a state recoverable by re-masking positions unavailable during generation, and enough usable positions with plausible tokens in both response directions. These are architectural conditions, not an evaluated universality claim; autoregressive-only or black-box systems need a different replayable perturbation.

Trust boundary. A hosted verifier assumes that the service runs the registered checkpoint and receives the document; a signed report neither proves execution nor protects privacy. Checkpoint and container hashes, append-only records, and independent execution can strengthen this arrangement. Public proof, attestation, service compromise, and adversarial requests are not evaluated.

Interpreting a detection result. A low p-value means only that the registered key and replay procedure find more checkpoint-consistent responses than expected by chance. It neither identifies the requester, attributes every passage, nor rules out editing. Reports should include the p-value, carrier count, key and checkpoint identifiers, precision, probe construction, and threshold.

6 Conclusion

ReplayMark encodes a keyed signal in reproducible model responses to hidden context. Selecting a carrier with a symmetric score before revealing the keyed direction yields an exact binomial test with a data-dependent carrier count. It preserves the deployed schedule and reaches 0.80 TPR on LLaDA and 0.90 on Dream without retuning, but needs 144 same-checkpoint calls per 512-token document; it therefore complements cheaper token-local detectors.

Limitations. The false-positive guarantee does not imply editing tolerance: random 10% re-denoising reduces TPR from 0.35 to 0.05 at $R = 1$ and from 0.88 to 0.50 at $R = 16$, while a contiguous span leaves earlier carriers intact. A PPL ratio below one does not imply better text, and the task cohorts are too small to interpret modest changes. Replay requires the same checkpoint and precision; evaluation covers two masked DLMs, one corpus, 16–50 documents per cohort, and provenance only.

References

[1] Scott Aaronson and Hendrik Kirchner. Watermarking GPT outputs, 2023. https://www.scottaaronson.com/talks/watermark.ppt.

[2] Marianne Arriola, Subham Sekhar Sahoo, Aaron Gokaslan, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Justin T. Chiu, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. In *ICLR*, 2025.

[3] Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces. In *NeurIPS*, 2021.

[4] Arka Bagchi, Akhil Bhimaraju, Moulik Choraria, Daniel Alabi, and Lav R. Varshney. Watermarking discrete diffusion language models. *arXiv preprint arXiv:2511.02083*, 2025.

[5] Mihir Bellare, Ran Canetti, and Hugo Krawczyk. Keying hash functions for message authentication. In *CRYPTO*, 1996.

[6] Heli Ben-Hamu, Itai Gat, Daniel Severo, Niklas Nolte, and Brian Karrer. Accelerated sampling from masked diffusion models via entropy bounded unmasking. In *NeurIPS*, 2025.

[7] Adam Block, Alexander Rakhlin, and Ayush Sekhari. GaussMark: A practical approach for structural watermarking of language models. In *ICML*, 2025.

[8] Ruibo Chen, Yihan Wu, Yanshuo Chen, Chenxi Liu, Junfeng Guo, and Heng Huang. A watermark for order-agnostic language models. In *ICLR*, 2025.

[9] Miranda Christ, Sam Gunn, and Or Zamir. Undetectable watermarks for language models. In *COLT*, 2024.

[10] Pierre Fernandez, Antoine Chaffin, Karim Tit, Vivien Chappelier, and Teddy Furon. Three bricks to consolidate watermarks for large language models. In *2023 IEEE International Workshop on Information Forensics and Security (WIFS)*, 2023.

[11] Jiayi Fu, Xuandong Zhao, Ruihan Yang, Yuansen Zhang, Jiangjie Chen, and Yanghua Xiao. GumbelSoft: Diversified language model watermarking via the GumbelMax-Trick. In *ACL*, 2024.

[12] Thibaud Gloaguen, Robin Staab, Nikola Jovanović, and Martin Vechev. Watermarking diffusion language models. In *ICLR*, 2026. arXiv:2509.24368.

[13] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *NeurIPS*, 2020.

[14] Pyo Min Hong and Albert No. dgMARK: Decoding-Guided Watermarking for Diffusion Language Models. In *ICML*, 2026. arXiv:2601.22985.

[15] Zhengmian Hu, Lichang Chen, Xidong Wu, Yihan Wu, Hongyang Zhang, and Heng Huang. Unbiased watermark for large language models. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[16] Nikola Jovanović, Robin Staab, and Martin Vechev. Watermark stealing in large language models. In *ICML*, 2024.

[17] Jaeyeon Kim, Kulin Shah, Vasilis Kontonis, Sham M. Kakade, and Sitan Chen. Train for the worst, plan for the best: Understanding token ordering in masked diffusions. In *ICML*, 2025.

[18] John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. A watermark for large language models. In *ICML*, 2023.

[19] John Kirchenbauer, Jonas Geiping, Yuxin Wen, Manli Shu, Khalid Saifullah, Kezhi Kong, Kasun Fernando, Aniruddha Saha, Micah Goldblum, and Tom Goldstein. On the reliability of watermarks for large language models. In *ICLR*, 2024.

[20] Kalpesh Krishna, Yixiao Song, Marzena Karpinska, John F. Wieting, and Mohit Iyyer. Paraphrasing evades detectors of AI-generated text, but retrieval is an effective defense. In *NeurIPS*, 2023.

[21] Rohith Kuditipudi, John Thickstun, Tatsunori Hashimoto, and Percy Liang. Robust distortion-free watermarks for language models. *TMLR*, 2024.

[22] Inception Labs, Samar Khanna, Siddhant Kharbanda, Shufan Li, Harshit Varma, Eric Wang, Sarah Birnbaum, Ziyang Luo, Yanis Miraoui, and Akash Palrecha. Mercury: Ultra-fast language models based on diffusion. *arXiv preprint arXiv:2506.17298*, 2025.

[23] Chia-Ming Lee, Shao-Kai Liu, Ming-Ching Chang, Xin Li, Yu-Lun Liu, and Chih-Chung Hsu. Commit locally, exit globally: Coordinating adaptive sampling and early exit in diffusion language models, 2026. URL https://arxiv.org/abs/2607.28166.

[24] Xiang Li, Feng Ruan, Huiyuan Wang, Qi Long, and Weijie J. Su. Robust detection of watermarks for large language models under human edits. *Journal of the Royal Statistical Society: Series B*, 2025.

[25] Yuqing Liang, Jiancheng Xiao, Wensheng Gan, and Philip S. Yu. Watermarking techniques for large language models: A survey. *Artificial Intelligence Review*, 2026.

[26] Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. In *ICML*, 2024.

[27] Eric Mitchell, Yoonho Lee, Alexander Khazatsky, Christopher D. Manning, and Chelsea Finn. DetectGPT: Zero-shot machine-generated text detection using probability curvature. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.

[28] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. In *NeurIPS*, 2025.

[29] Jingyang Ou, Shen Nie, Kaiwen Xue, Fengqi Zhu, Jiacheng Sun, Zhenguo Li, and Chongxuan Li. Your absorbing discrete diffusion secretly models the conditional distributions of clean data. In *ICLR*, 2025.

[30] Fabien A. P. Petitcolas, Ross J. Anderson, and Markus G. Kuhn. Information hiding: A survey. In *Proceedings of the IEEE*, 1999.

[31] Ohad Raban, Ethan Fetaya, and Gal Chechik. LR-DWM: Efficient watermarking for diffusion language models. *arXiv preprint arXiv:2601.12376*, 2026.

[32] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

[33] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.

[34] Subham Sekhar Sahoo, Marianne Arriola, Aaron Gokaslan, Edgar Marroquin Marroquin, Alexander M. Rush, Yair Schiff, Justin T. Chiu, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. In *NeurIPS*, 2024.

[35] Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis Titsias. Simplified and generalized masked diffusion for discrete data. In *NeurIPS*, 2024.

[36] Qingyan Wei, Yaojie Zhang, Zhiyuan Liu, Pengfei Zeng, Yu Wang, Biao Qi, Dong Liu, and Linfeng Zhang. Accelerating diffusion large language models with SlowFast sampling: The three golden principles. In *ICLR*, 2026.

[37] Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. Fast-dLLM: Training-free acceleration of diffusion LLM by enabling KV cache and parallel decoding. In *ICLR*, 2026.

[38] Linyu Wu, Linhao Zhong, Wenjie Qu, Yifan Li, Yue Liu, Shengfang Zhai, Chunhua Shen, and Jiaheng Zhang. DMark: Order-agnostic watermarking for diffusion large language models. *arXiv preprint arXiv:2510.02902*, 2025.

[39] Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7B: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025.

[40] Xuandong Zhao, Yu-Xiang Wang, and Lei Li. Protecting language generation models via invisible watermarking. In *ICML*, 2023.

[41] Xuandong Zhao, Prabhanjan V. Ananth, Lei Li, and Yu-Xiang Wang. Provable robust watermarking for AI-generated text. In *ICLR*, 2024.

[42] Fengqi Zhu, Rongzhen Wang, Shen Nie, Xiaolu Zhang, Chunwei Wu, Jun Hu, Jun Zhou, Jianfei Chen, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. LLaDA 1.5: Variance-reduced preference optimization for large language diffusion models. *arXiv preprint arXiv:2505.19223*, 2025.

A Additional Tables

Table 3 reports the measured per-key false-positive rate of the deployed decision rule, and Figure 5 plots the same calibration on human-written text: empirical against nominal FPR over 1800 C4 documents under three keys. Table 4 shows how detection and verification cost trade off as the number of probe patterns varies, and Table 5 summarises the post-editing evaluation under the deployed pooled detector.

Nominal α	Mean FPR	Highest per-key FPR
0.10	0.0121	0.0583
0.05	0.0037	0.0250
0.01	0.0000	0.0000

Table 3: **Observed false-positive rates across keys.** The evaluation uses 20 keys and 120 unwatermarked generations per key under the deployed decision rule.

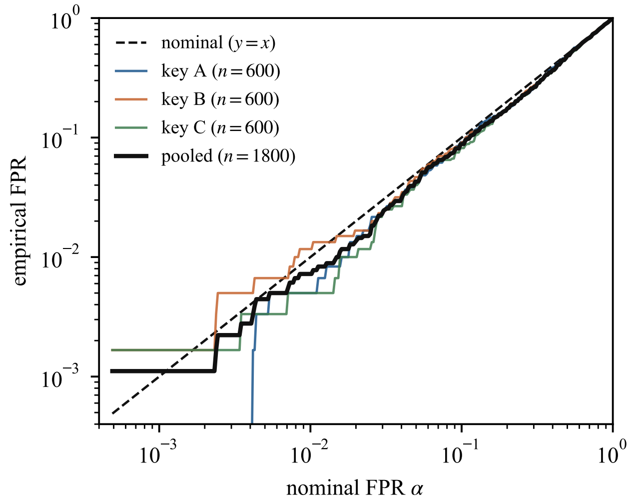

Figure 5: **Empirical against nominal false-positive rate on human-written text.** 600 C4 documents per key under three keys (pooled $n = 1800$), scored by the deployed verifier; the diagonal marks ideal equality. Empirical rates track the nominal level across three orders of magnitude and sit at $0.043/0.0072/0.0011$ at the $0.05/0.01/0.001$ operating points.

L	Verify calls ↓	TPR at FPR ↑			Sync rate	PPL ratio
		5%	1%	0.1%		
2	48	0.73	0.57	0.50	0.629	1.01×
4	80	0.77	0.70	0.67	0.650	0.86×
8	144	0.77	0.77	0.73	0.658	0.99×

Table 4: **Fewer probe patterns make verification cheaper.** Each row halves the number of probe patterns L; verification replays $16(L+1)$ model calls. Detection is measured at $R = 16, \kappa = 0.05$ on LLaDA, 512 tokens, $n = 30$ fresh prompts. Going from $L = 8$ to $L = 4$ keeps most of the detection at a little over half the cost; going to $L = 2$ saves little more and loses about a tenth of TPR at every operating point. PPL ratios rest on the 14–15 pairs surviving the decoded-length rule; the shared control's false positives stay at or below nominal for every L (0 of 30 at 0.1%).

Edit	Strength	TPR at FPR ↑	
		1%	0.1%
None	n/a	0.88	0.88
Re-denoising (random positions)	5% of positions	0.88	0.75
	10% of positions	0.50	0.38
Re-denoising (contiguous span)	10% of positions	0.88	0.88
	30% of positions	0.88	0.88
	50% of positions	0.81	0.69
	75% of positions	0.50	0.31
Paraphrase (Qwen2.5-7B, full document)	n/a	0.00	0.00

Table 5: **Detection after editing.** ReplayMark uses $R = 16, \kappa = 0.05$ on 1024-token documents ($n = 16$). The pooled detector uses the same analytic thresholds as Table 1. Random-position edits disrupt later prefixes, whereas a contiguous span preserves earlier carriers. One document changes TPR by about 0.06.

B Per-Document Carrier Statistics

Figure 6 reports per-document statistics from verifier replay over the saved 512-token outputs. Carrier counts average 271 (LLaDA control), 192 (LLaDA R=8, κ=0.1), 214 (Dream control) and 208 (Dream R=16, κ=0.05), with wide spread ($\approx$ 30 to 500) driven by early termination, since EOS padding admits few carriers. Most per-document Wilson intervals lie above 0.5; the weakest overlap it.

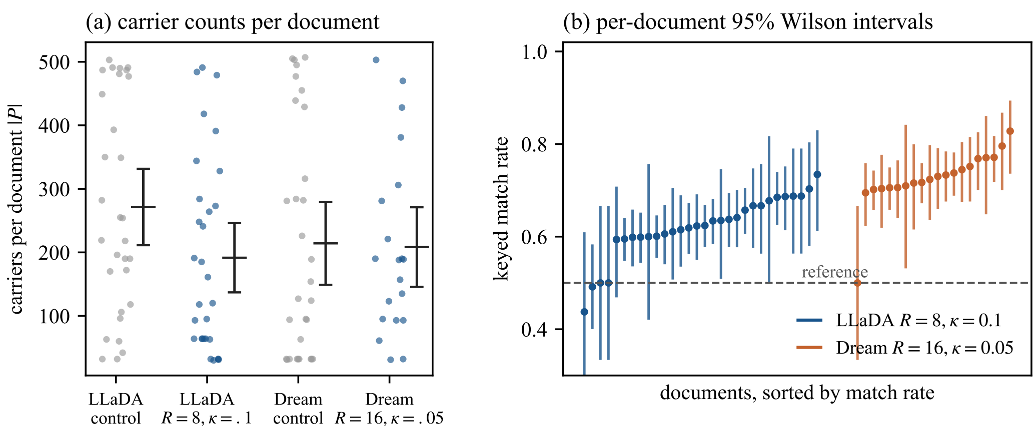

Figure 6: **Per-document carrier counts and detection intervals.** (a) Carrier pool size $|P|$ per document for the control and watermarked outputs of both models (dot = document; whisker = mean $\pm$ 95% CI). Counts range from about 30 to 500: short, EOS-padded documents admit few carriers. (b) Per-document 95% Wilson intervals on the keyed match rate for the watermarked outputs, sorted. Most intervals lie above the 0.5 reference rate; the weakest overlap it.